COMP 551 - MiniProject 2 Report (Group 13)

Members: Mark Maroun (261115147) Louis Stalet (261079468) Rami El-Nawam (261087427)

1 Abstract

This project studies binary email classification using the UCI Spambase dataset and logistic regression trained with mini-batch stochastic gradient descent (SGD). We first preprocess the dataset by holding out a small training subset and stan- dardizing continuous input features using statistics computed from the training data. Using this reduced training set, we implement logistic regression from scratch and train the model using mini-batch SGD with an L2 regularization term applied to the weights. Model performance is evaluated using cross-entropy loss and classification accuracy. We then apply K-fold cross-validation on the training portion to tune key optimization hyperparameters such as learning rate, batch size, and number of training epochs. Finally, we examine the effect of L2 regularization strength by sweeping the parameter λ and computing both training and validation per- formance across folds. This allows us to illustrate the bias–variance trade-off in a setting where overfitting is expected due to limited training data, and to identify a regularization level that improves generalization. The selected hy- perparameters are then used to retrain the model on the full training set and evaluate performance on the held-out test set.

2 Introduction

The UCI Spambase dataset consists of email messages represented by 57 contin- uous features derived from word and character frequencies, along with a binary label indicating whether the email is classified as spam or not. In this project, we model the probability that an email is spam using logistic regression trained with mini-batch stochastic gradient descent (SGD).

To create a setting where overfitting is clearly observable, we hold out only 5% of the dataset for training and reserve the remaining 95% as a test set for final evaluation. This results in a small training set of approximately 230 sam- ples, making regularization necessary for improving generalization. All input features are standardized using statistics computed on the training set in order to improve numerical stability during optimization.

3 Methods

We implement logistic regression for binary classification and train the model using mini-batch stochastic gradient descent (SGD). Given an input feature vector $x_i \in \mathbb{R}^{57}$, the model estimates the probability that an email belongs to the spam class using the sigmoid function:

$$\hat{y}_i = \sigma(w^\top x_i + b),$$

where w represents the weight vector and b is a scalar bias term.

The model is trained by minimizing the binary cross-entropy loss over the training set. To improve generalization in the small-sample regime, we include an L2 regularization penalty applied only to the weight parameters. We use the convention:

$$\mathcal{L}_{reg}(w, b) = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\right] + \frac{\lambda}{2}\|w\|_2^2,$$

where λ controls the strength of regularization. Under this convention, the gradient with respect to the weights includes an additional term λw, while the bias parameter remains unpenalized.

Training is performed using mini-batches of size B. At each update step, gradients are computed on the current batch and the parameters are updated according to:

$$w \leftarrow w - \eta \left(\frac{1}{B} X^\top (\hat{y} - y) + \lambda w \right), \qquad b \leftarrow b - \eta \left(\frac{1}{B} \sum (\hat{y} - y) \right),$$

where η denotes the learning rate. For numerical stability, we clip the input to the sigmoid and clip predicted probabilities away from 0 and 1 when computing the loss.

Prior to training, the dataset is shuffled using a fixed random seed and split into a 5% training set and a 95% held-out test set. All input features are standardized using the mean and standard deviation computed from the training set only, and the same transformation is applied to the test set to avoid data leakage.

For hyperparameter tuning, we use K-fold cross-validation on the training portion of the data (we use $K = 5$). For each candidate hyperparameter configuration, the model is trained on $K - 1$ folds and evaluated on the remaining fold, and the resulting cross-entropy loss and accuracy are averaged across folds. This procedure is used to select optimization parameters (Task 2) and to evaluate the effect of different regularization strengths (Task 3).

4 Results

4.1 Task 1: Learning Curves (No Regularization vs. L2)

Preprocessing. We load the Spambase dataset, shuffle using a fixed random seed (`RANDOM_SEED=2026`), and split the data into a small training set (5%) and a held-out test set (95%). We standardize all 57 features using the training-set mean and standard deviation, and apply the same transformation to the test set.

Training curves and stability. Figure 1 shows the training cross-entropy loss over epochs for both the unregularized model ($\lambda = 0$) and an L2-regularized model (fixed λ). In both cases the loss decreases steadily across epochs, indicating stable optimization for the chosen learning rate and batch size. The L2 curve typically remains slightly higher than the unregularized curve on the training set due to the added penalty term, which discourages large weights and therefore reduces the model's capacity to overfit the limited training data.

See figure 1 "Task 1: Training cross-entropy loss vs. epochs for no regularization ($\lambda = 0$) and L2" in the appendix

SGD vs. mini-batch discussion. With such a small training set, batch size changes affect the noise level of gradient estimates. In general, batch size $B = 1$ (pure SGD) produces noisier updates and less smooth loss curves, while moderate mini-batches (e.g., $B = 16$) reduce variance in the gradient and yield smoother training dynamics. In our implementation, the mini-batch setting provides stable convergence over 200 epochs and produces a clear comparison between the regularized and unregularized objectives.

4.2 Task 2: Hyperparameter Tuning with K-Fold CV

We perform K-fold cross-validation (with $K = 5$) on the training set to tune learning rate η, batch size B, and number of epochs. For each hyperparameter triple (η, B, epochs), we train on $K - 1$ folds and compute validation cross-entropy and validation accuracy on the held-out fold, then average results across folds.

Summary of CV results. Table 1 shows a compact summary of the grid search. (You can expand this table with more rows if you want to show more of the grid; this version is meant to be readable while still showing the main outcomes.)

Table 1: Task 2: Example summary of cross-validation results across selected hyperparameter settings (average across 5 folds).

Learning rate η	Batch size B	Epochs	Mean CV loss	Mean CV acc.
0.0001	1	200	0.3301	0.9087
0.001	1	200	0.2758	0.9304
0.01	16	200	0.2752	0.9261
0.1	32	50	**0.2751**	**0.9304**
1.0	1	100	2.5550	0.8261

Chosen configuration. Based on minimizing mean cross-entropy loss (and noting that accuracy is also competitive), the best configuration found in this grid is:

$$\eta = 0.1, \quad B = 32, \quad \text{epochs} = 50,$$

with mean CV loss ≈ 0.2751 and mean CV accuracy ≈ 0.9304.

Reflection on which hyperparameters mattered most. The learning rate had the strongest impact. Very small learning rates required more epochs to approach low loss, while very large learning rates (e.g., $\eta = 1$ with $B = 1$) led to unstable behavior and much worse validation loss. Batch size also mattered in terms of training stability: moderate mini-batches typically reduced variance in the updates compared to $B = 1$, and helped the model converge reliably. The number of epochs mattered mainly when the learning rate was small; with sufficiently large learning rates, performance often saturated by 50–100 epochs.

4.3 Task 3: Bias–Variance Trade-Off via λ Sweep

To illustrate bias–variance behavior, we sweep the L2 regularization strength over a logarithmic grid and evaluate mean training and validation cross-entropy using 5-fold CV. We fix optimization hyperparameters to settings that allow the model to fit the training set well and expose overfitting in this small-data regime (learning rate, batch size, and epochs held constant). For each λ, we compute the mean training loss and mean validation loss across folds.

Figure 2 plots training and validation loss versus λ on a log scale. For very small λ, the model fits the training data strongly (low training loss), but validation loss is higher relative to the best point on the curve, indicating higher variance. As λ increases, training loss rises (reduced flexibility), while validation loss initially improves and then worsens again at large λ as the model begins to underfit (high bias). This produces the expected U-shaped validation curve, with best generalization at an intermediate regularization strength.

See figure 3 "Task 3: Mean training and validation cross-entropy vs. L2 regularization strength λ (5-fold CV). The x-axis is log-scaled." in the appendix

Selected λ and final test evaluation. We select the λ value that minimizes mean validation cross-entropy under CV, retrain the model on the full training split using this selected λ, and then report cross-entropy and accuracy on the held-out test set. We select the regularization strength λ=0.0001 as it achieves the lowest mean validation cross-entropy under 5-fold cross-validation. The model is then retrained on the full training split using this selected value and evaluated on the held-out test set. This results in a final test cross-entropy loss of 0.3146 and a classification accuracy of 0.9012.

4.4 Task 4: L1-Regularized Logistic Regression and the Regularization Path

See figure 4 "Task 4: L1 regularization path plot. The x-axis is log-scaled." in the appendix

A clear explanation of what the regularization path represents. The regularization path represents the coefficient value for each variable as we increase C. Since $C = 1/\lambda$, a larger C represents weaker regularization, and a smaller C means stronger regularization. When C is small, as we can see in the plot, all of the coefficient values are equal to 0. As C increases and the regularization becomes weaker, the coefficients start to take on values. As C grows, these values become larger. Finally, when C is very large, all the coefficients are non-zero, and some of the values that appeared at a smaller C become very large. This regularization path shows the most important features.

How sparsity changes as regularization changes. Let's define sparsity: in this context, it represents the number of features that are non-zero. As the regularization strength decreases (meaning C becomes larger), the number of non-zero features—and thus the sparsity—increases. For example,from figure 4 when $\lambda = 100$ (or $C = 10^{-2}$), we have a sparsity of 0. When $\lambda = 10$, we have a sparsity of around 20, and for $\lambda = 1$, we have a sparsity of around 40. As λ continues to decrease, the sparsity continues to increase until all features are included in the model.

Table 2: Task 4: Most important features identified by the first emergence in the L1 regularization path (at $C \approx 0.03$).

Feature Index	Initial Coefficient Value
6	0.060323
20	0.055063
23	0.001254
51	0.163656
52	**0.213759**

Which features appear most important. Based on the results in the provided table 2 and the figure 3, we have determined that the first value of C where coefficients begin to become non-zero is 0.03039195382313198. At this specific point, we identified the first set of features to emerge: features 6, 20, 23, 51, and 52. Among these initial features, the most important coefficient appears to be the 52nd one, as it possesses the highest absolute value. As noted in the data, coefficient 52 has an initial value of 0.213759. Other highly significant predictors include feature 51, with an initial value of 0.163656, and coefficient 6, which starts at 0.060323. Because these features are the only one remaining under heavy regularization,they are considered the most important ones.

Trade-off between interpretability (sparsity) and performance. From the figures 4 and 5, we can understand the trade-off between interpretability and performance. As seen in the CV performance vs. C plot, performance reaches its maximum at $C = 0.9$ with a value of approximately 0.88. However, looking at the Sparsity plot for that same C value, we see that nearly all variables are non-zero, which makes the model very difficult to interpret. In contrast, at $C = 0.1$, the performance is approximately 0.85, but the model uses only about 20 non-zero variables. This means that if we are willing to give away 0.03 points of performance, we can use less than half of the variables, which significantly helps with interpretability. Our goal is to find the perfect ratio between interpretability and performance to create the best model that remains the easiest to read and understand.

5 Discussion and Conclusion

In this project, we implemented mini-batch stochastic gradient descent (SGD) for logistic regression with L_2 regularization. We conducted experiments to analyze the bias–variance trade-off. Subsequently, we implemented K-fold cross-validation for hyperparameter tuning in order to select the best-performing model. Finally, we visualized the regularization path of an L_1-regularized logistic regression model.

This analysis clearly illustrates the trade-off between bias and variance, particularly evident in Figure

2. When applying a small regularization parameter, the model fits the training data exceptionally well. However, the large gap between the training and validation losses indicates low bias but very high variance. In contrast, under very large regularization, we observe similarly high cross-entropy loss for both the training and validation sets. This scenario effectively eliminates variance but introduces substantial bias. These results highlight the importance of identifying an optimal balance between bias and variance in order to develop a model that generalizes well.

The L_1 and L_2 regularization techniques behave differently. While L_2 regularization penalizes the squared magnitude of the weights to prevent any single feature from becoming too large, the model typically retains all 57 features. In contrast, L_1 regularization penalizes the absolute values of the weights. As observed in Figure 3, this causes many feature coefficients to shrink exactly to zero. Therefore, L_1 regularization serves as an effective method for feature selection, allowing us to identify the most important variables in the model.

Regarding the limitations of our study, we trained our model on only 5% of the total dataset. As a result, the model had limited data to learn the underlying patterns effectively, which likely impacted both the hyperparameter tuning process and the bias–variance analysis. Furthermore, in Task 3, we kept the optimization hyperparameters fixed at $\eta = 0.1$ and $B = 16$. Consequently, we did not perform the bias–variance comparison using the optimal hyperparameter configuration, which may have influenced our findings regarding the best trade-off point.

For future work, we plan to increase the size of the training dataset to enable the model to better capture the underlying data structure. Additionally, we could explore alternative optimization algorithms, such as Adam or RMSProp, to improve convergence speed and overall optimization performance.

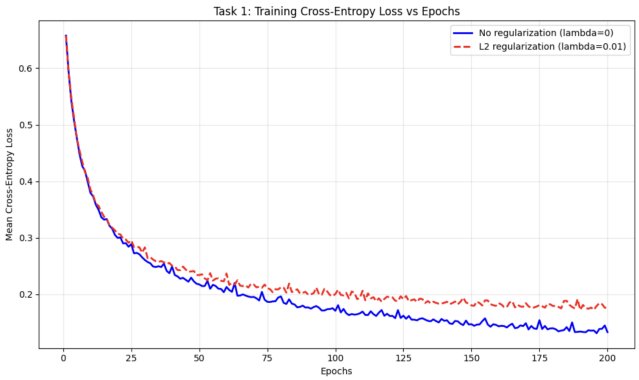
6 Statement of Contributions

The team met and decided on the task split as follows: Louis Stalet handled Tasks 1, 2, and 3; Mark Maroun handled Task 4, the Task 4 results, and the conclusion of the report; and Rami El-Nawam wrote the introduction, methods, and results for Tasks 1, 2, and 3 in the report. All members reviewed the final submission and discussed the results and conclusions.

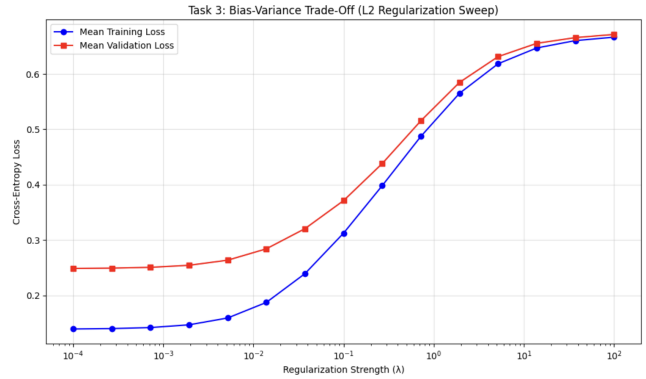
7 Statement on the Use of LLMs and Other Resources

We used ChatGPT to help reformat and polish the report into LATEX and to improve clarity. We did not use it to generate code or derive results.

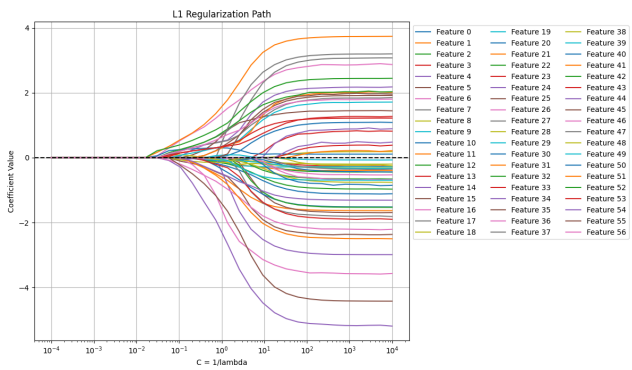
Appendix



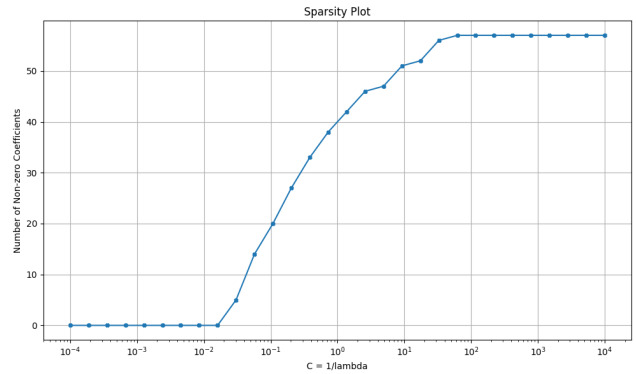
(a) Figure 1: Task 1: Training cross-entropy loss vs. epochs for no regularization ($\lambda = 0$) and L2 regularization (fixed λ).



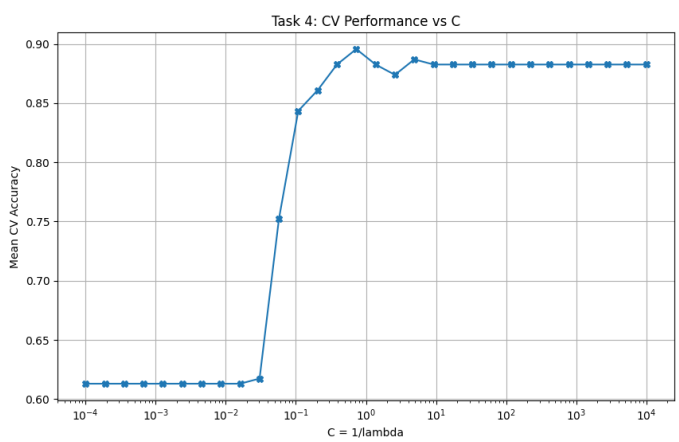
(b) Figure 2: Task 3: Mean training and validation cross-entropy vs. L2 regularization strength λ (5-fold CV). The x-axis is log-scaled.



(c) Figure 3: Task 4: L1 regularization path plot. The x-axis is log-scaled.



(d) Figure 4: Task 4: Sparsity plot showing the number of non-zero coefficients versus C. The x-axis is log-scaled.



(e) Figure 5: Task 4: Mean cross-validation accuracy vs. C. The x-axis is log-scaled.